Das Paper verstehen

KI-Coding-Agenten in Entwickler-Workflows: Gestaltungsprinzipien und Evidenzgrenzen

Lern- und Verständnisguide zur Präsentation und fachlichen Abfrage

Stand des Papers: Literatur bis 16. Oktober 2026

Das ganze Paper in fünf Sätzen

1. KI-Coding-Agenten sind in der Praxis angekommen: In rund jedem fünften untersuchten GitHub-Projekt finden sich Spuren davon.
2. Wir haben 21 Studien von 2024 bis 2026 gelesen und gefragt: *Wie sollte man Agenten in die Arbeit einbinden, und was ist dazu wirklich belegt?*
3. Daraus haben wir fünf Prinzipien abgeleitet: **Planung** gemeinsam prüfen, **Autonomie** nach Risiko staffeln, **Kontext** gezielt ergänzen, mehrstufig **verifizieren**, überprüfbar **integrieren**.
4. Belegt sind nur einzelne, eng begrenzte Maßnahmen. Dass die fünf Prinzipien zusammen wirken, hat niemand getestet, und ein allgemeiner Produktivitätsgewinn folgt aus der Literatur nicht.
5. Wer Agenten bewertet, muss getrennt messen: aktive Arbeitszeit, Gesamtdauer, Testbestehen und echte Korrektheit.

Inhaltsverzeichnis

1 Worum geht es überhaupt? 3
- 1.1 Das Problem . . . 3
- 1.2 Die Forschungsfrage . . . 3
- 1.3 Die drei Blickwinkel (Dimensionen) . . . 3
- 1.4 Der Gedankengang des Papers . . . 4

2 Grundbegriffe, die man kennen muss 4
- 2.1 Was ist ein Coding-Agent genau? . . . 4
- 2.2 SWE-bench: der Standardtest für Agenten . . . 4
- 2.3 Begriffe aus Statistik und Studiendesign . . . 5
- 2.4 Weitere Fachbegriffe . . . 6

3 Das Paper Abschnitt für Abschnitt 6
- 3.1 Einleitung: Warum das Thema wichtig ist . . . 6
- 3.2 Vorgehen: Wie wir gearbeitet haben . . . 6
- 3.3 Befunde A: Autonomie, Mensch und Produktivität . . . 7
- 3.4 Befunde B: Kontext und Steuerung . . . 7
- 3.5 Befunde C: Verifikation, Sicherheit und Integration . . . 8

3.6 Die fünf Gestaltungsprinzipien . . . 9
3.7 Diskussion: Was trägt die Evidenz? . . . 10
3.8 Fazit . . . 11

4 Die wichtigsten Studien verständlich erklärt 11
4.1 Motivation und Hintergrund . . . 11
4.2 Produktivität . . . 12
4.3 Kontext . . . 13
4.4 Verifikation und Integration . . . 13
4.5 Weitere Arbeiten im Korpus (kurz) . . . 14

5 Wie hängen Befunde und Prinzipien zusammen? 15

6 Typische Denkfehler und wie man es richtig sagt 15

7 Prüfungsfragen mit Antworten 15

8 Alle wichtigen Zahlen auf einen Blick 18

1 Worum geht es überhaupt?

1.1 Das Problem

Früher haben KI-Werkzeuge wie GitHub Copilot vor allem *Code vervollständigt*: Man tippt, die KI schlägt die nächste Zeile vor. Ein **Coding-Agent** geht weiter. Man gibt ihm eine ganze Aufgabe („Behebe diesen Bug“), und er

- sucht selbst die relevanten Dateien,
- plant die Änderung,
- ändert Code, führt Befehle und Tests aus,
- schaut sich das Ergebnis an und macht weiter, bis er fertig ist.

Dadurch verändert sich die Arbeit der Entwickler: Weniger selbst tippen, mehr *beschreiben, überwachen und prüfen*. Genau hier entsteht die Frage des Papers: Spart das wirklich Zeit? Und ist der Code dann korrekt?

Anschaulich erklärt

Copilot ist wie ein Beifahrer, der dir beim Schreiben das nächste Wort einflüstert. Ein Agent ist wie ein Praktikant, dem du eine Aufgabe gibst und der sie allein erledigt. Der Praktikant spart dir Tipparbeit, aber du musst ihm die Aufgabe gut erklären, zwischendurch nachsehen und am Ende sein Ergebnis kontrollieren. Ob du am Ende Zeit sparst, hängt davon ab, wie viel Erklären und Kontrollieren kostet.

1.2 Die Forschungsfrage

Forschungsfrage

Welche Gestaltungsprinzipien für Entwickler-Workflows mit KI-Coding-Agenten lassen sich aus aktueller Literatur ableiten, und welche empirischen Hinweise und Evidenzgrenzen bestehen hinsichtlich Codequalität und Produktivität?

Die Frage hat **zwei Teile**, und das ist wichtig für das Fazit:

1. **Ableiten:** Was sollte man tun? (Ergebnis: fünf Prinzipien)
2. **Prüfen:** Wie gut ist das belegt? (Ergebnis: nur Einzelmaßnahmen belegt, der Gesamtworkflow nicht)

1.3 Die drei Blickwinkel (Dimensionen)

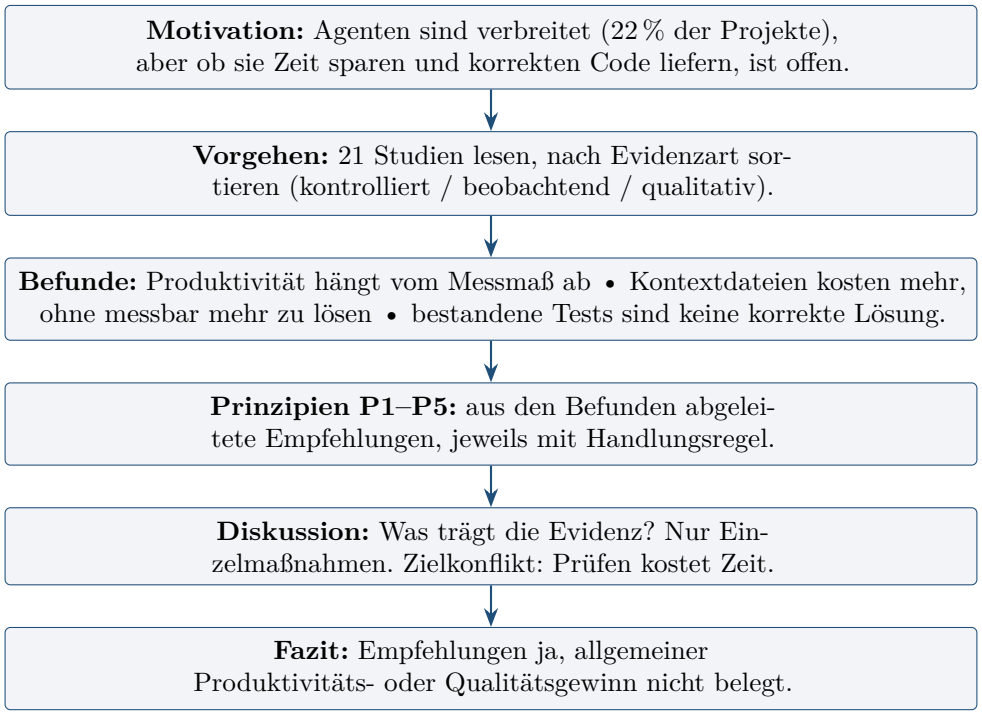
Das Paper sortiert alle Studien nach drei Fragen:

Autonomie Wie viel darf der Agent allein entscheiden, und wo greift der Mensch ein?

Kontext Welches Wissen über das Projekt bekommt der Agent (z. B. über eine `CLAUDE.md`)?

Qualitätssicherung Wie prüft man, ob das Ergebnis stimmt (Tests, Linter, Review, Sicherheit)?

1.4 Der Gedankengang des Papers



2 Grundbegriffe, die man kennen muss

2.1 Was ist ein Coding-Agent genau?

Im Paper (nach Liu et al.) gilt als Coding-Agent ein **System auf Basis eines großen Sprachmodells (LLM), das Entwicklungsaktionen mit Werkzeugen ausführt und die Rückmeldungen der Umgebung für die nächsten Schritte nutzt**. Entscheidend ist also die Schleife *handeln → Ergebnis ansehen → weiter handeln*. Liu et al. beschreiben vier Bausteine: Planung, Gedächtnis, Wahrnehmung und Aktion.

Der **Autonomiegrad** kann sehr unterschiedlich sein:

- **Agentless**: fester Ablauf in drei Schritten (Fehler finden, reparieren, prüfen). Das Modell entscheidet nicht selbst, was als Nächstes kommt.
- **HULA**: Der Mensch ist fest eingeplant. Er prüft zuerst den Plan des Agenten, dann den Code.
- **OpenHands**: offene Plattform, auf der der Agent recht frei in einer abgeschotteten Umgebung (Docker-Sandbox) arbeitet.

2.2 SWE-bench: der Standardtest für Agenten

Fast alle technischen Studien messen ihre Agenten auf **SWE-bench**. Der Benchmark besteht aus echten Bug-Meldungen (Issues) aus 12 bekannten Python-Projekten auf GitHub. Der Agent bekommt den Issue-Text und den Code und soll einen Patch schreiben. Als „gelöst“ zählt die Aufgabe, wenn die Tests aus der echten Korrektur des Projekts bestehen.

- **SWE-bench** (voll): 2.294 Aufgaben

- **SWE-bench Lite**: 300 Aufgaben, kleiner und günstiger
- **SWE-bench Verified**: 500 Aufgaben, von Menschen geprüft

Vorsicht, Denkfehler

„Gelöst“ heißt bei SWE-bench nur: *Die ausgewählten Tests sind grün.* Das ist nicht dasselbe wie „der Code ist richtig“. Genau darauf baut ein großer Teil des Papers auf (Abschnitt 3.5).

2.3 Begriffe aus Statistik und Studiendesign

Begriff	Bedeutung
Kontrollierte Studie	Man vergleicht zwei Bedingungen unter sonst gleichen Umständen (z. B. mit und ohne Agent). Nur so kann man einen Unterschied auf die Bedingung zurückführen.
RCT	*Randomized Controlled Trial*: Wer welche Bedingung bekommt, entscheidet der Zufall. Gilt als stärkstes Design.
Within-Subject	Jede Person probiert beide Bedingungen aus (bei Chen: jeder nutzt Copilot *und* OpenHands).
Ablation	Man baut ein einzelnes Teil aus (z. B. den Linter) und misst, was sich ändert. So isoliert man dessen Wirkung.
Deskriptiv / Mining	Man beobachtet nur, was es gibt (z. B. GitHub-Daten durchsuchen). Zeigt Zusammenhänge, aber keine Ursachen.
Qualitativ	Interviews, Aussagen, Muster. Zeigt *wie* etwas gemacht wird, nicht *wie oft* oder *wie wirksam*.
Endpunkt	Die Größe, die gemessen wird: Erfolgsrate, aktive Zeit, Gesamtdauer, Kosten, Anzahl PRs …
Signifikant	Der Unterschied ist so groß, dass Zufall als Erklärung unwahrscheinlich ist.
Nicht signifikant	Man kann den Unterschied nicht vom Zufall trennen. **Das heißt nicht, dass beide gleich gut sind!**
Standardfehler	Wie unsicher eine Schätzung ist (Cui: 26,08 % bei einem Standardfehler von 10,3 %).
Prozentpunkte	Unterschied zwischen zwei Prozentwerten: 25 % → 60 % sind **+35 Prozentpunkte** (relativ wären das +140 %).
@1 / @3	Erfolg im ersten Versuch bzw. in mindestens einem von drei Versuchen.
Preprint	Veröffentlicht, aber noch nicht von Fachgutachtern geprüft (meist auf arXiv).

2.4 Weitere Fachbegriffe

Kontextdatei	Datei im Repository (`CLAUDE.md`, `AGENTS.md`), in der steht, was der Agent über das Projekt wissen soll: Konventionen, Architektur, Build- und Testbefehle.
Context Engineering	Steuern, *welche* Informationen das Modell bekommt (im Gegensatz zum Prompt Engineering: *wie* man die Aufgabe formuliert).
Linter	Programm, das Code automatisch auf Syntax- und Stilfehler prüft, ohne ihn auszuführen.
Testsuite	Alle Tests eines Projekts. SWE-bench führt nur einen Teil davon aus.
Differenzielles Testen	Man erzeugt Tests, die gezielt Unterschiede zwischen zwei Lösungen (Agent vs. Referenz) aufdecken.
Pull Request (PR)	Vorschlag einer Codeänderung, der geprüft und dann ins Projekt übernommen („gemergt“) wird.
Sandbox	Abgeschottete Umgebung. Der Agent kann darin nichts am echten System kaputt machen.
Human-in-the-Loop	Der Mensch ist an festen Stellen beteiligt (z. B. Plan freigeben).
Automation Bias	Man vertraut automatischen Ergebnissen zu sehr und prüft sie zu wenig.

3 Das Paper Abschnitt für Abschnitt

3.1 Einleitung: Warum das Thema wichtig ist

- Robbes et al. haben 128.018 GitHub-Projekte untersucht: **22,20 %** nutzen Coding-Agenten (vorsichtige Schätzung), bis zu **28,66 %** (hochgerechnet). Die Stichprobe steht aber nicht für die gesamte Softwareentwicklung.
- Die Arbeit verschiebt sich zum Spezifizieren, Überwachen und Prüfen (Dhanorkar et al.).
- Ob Agenten Zeit sparen und korrekten Code liefern, kann man **weder aus Benchmark-Erfolgen noch aus der Zahl der PRs** ablesen.

Beitrag des Papers: eine vergleichende Zusammenfassung (Synthese) entlang von Autonomie, Kontext und Qualitätssicherung. Die Prinzipien sind *begründete Empfehlungen*, ihre gemeinsame Wirkung wird *nicht* als bewiesen vorausgesetzt.

3.2 Vorgehen: Wie wir gearbeitet haben

1. **Suche:** ACM Digital Library, IEEE Xplore und arXiv, dazu „Rückwärtssuche“ (in den Literaturverzeichnissen der gefundenen Arbeiten weitersuchen).
2. **Suchbegriffe:** *coding agent, software engineering agent, agentic coding, LLM-based agent* kombiniert mit *workflow, human-in-the-loop, autonomy, code review, testing, productivity, code quality.*
3. **Auswahl:** Duplikate raus, dann Titel/Abstract prüfen, dann Volltext. Aufgenommen: ab 2024, Englisch oder Deutsch, mehrschrittig arbeitende, werkzeugnutzende LLM-Agenten, empirisch untersucht oder als System mit Evaluation, mit Bezug zu Workflow, Qualität oder Produktivität.

4. **Auswertung:** Für jede Studie Design, Werkzeug, Population/Benchmark, Endpunkt und Einschränkungen gegenübergestellt.

Merke

Das Paper trennt drei **Evidenzarten**: kontrollierte Vergleiche, deskriptive Beobachtungen und qualitative Aussagen. Es rechnet **keine Effekte über Studien hinweg zusammen**, weil die Studien Verschiedenes messen. Beispiel aus dem Paper: Vergleicht man zwei Agenten, die sich gleichzeitig in Modell, Werkzeugen und Budget unterscheiden, weiß man nicht, ob der Unterschied an der Autonomie liegt.

Ehrliche Einschränkung: Es ist eine *narrative* Analyse, keine *systematische* Vollerhebung, weil die Trefferzahlen der einzelnen Schritte nicht berichtet werden.

3.3 Befunde A: Autonomie, Mensch und Produktivität

Kernaussage: **Weniger aktive Arbeit heißt nicht kürzere Gesamtdauer.** Und: Verschiedene Studien messen Produktivität völlig unterschiedlich.

Studie	Wer und womit?	Ergebnis	Warum vorsichtig?
Chen et al.	20 Studierende; OpenHands vs. Copilot	Erfolg 25 → 60 %; aktive Zeit 25,1 → 12,5 min	Mit Laufzeit des Agenten 27,9 min; kleine Gruppe
Cui et al.	4.867 Entwickler in Firmen; Copilot-*Vervollständigung*	+26,08 % erledigte Aufgaben (PRs/Woche)	Kein Agent!
Becker et al.	16 erfahrene Open-Source-Entwickler, eigene Projekte; meist Cursor	19 % *länger* mit KI; geschätzt 20 % schneller	Kleine Gruppe, Werkzeuge Anfang 2025

Anschaulich erklärt

Bei Chen sitzt die Person mit Agent nur 12,5 Minuten aktiv an der Aufgabe statt 25,1. Aber der Agent arbeitet zwischendurch allein, und zusammen dauert es 27,9 Minuten. Das ist wie beim Geschirrspüler: Du arbeitest weniger als beim Abwaschen von Hand, aber fertig ist das Geschirr nicht schneller. Ob das ein Gewinn ist, hängt davon ab, ob du die Wartezeit sinnvoll nutzen kannst.

Warum Cui (+26 %) und Becker (+19 % Zeit) sich nicht widersprechen: anderes Werkzeug (Vervollständigung vs. KI-Werkzeuge mit Chat/Agent), andere Personen (breite Belegschaft vs. Experten im eigenen Projekt), anderes Messmaß (Anzahl PRs vs. Bearbeitungszeit). Lehre: Werkzeugtyp, Population und Zeitmaß muss man beachten.

Menschliche Aufsicht ist nicht ein einzelner Moment. Dhanorkar et al. unterscheiden aus 17 Interviews vier Formen: *Vorgaben vorab* (z. B. Berechtigungen), *gemeinsame Planung*, *laufende Überwachung* und *nachträgliches Review*. Sie beschreiben auch, dass Entwickler dem Plan des Agenten als Prüfhilfe vertrauen, aber: **Ein freigegebener Plan beweist keine korrekte Umsetzung.**

3.4 Befunde B: Kontext und Steuerung

- **Was steht in Kontextdateien?** Chatlatanagulchai et al. (253 `CLAUDE.md`-Dateien): vor allem Build- und Run-Befehle, Implementierungsdetails, Architektur.

- **Wie verbreitet?** Mohsenimofidi et al.: in 466 von 10.000 Open-Source-Projekten (knapp 5 %). Gemessen wurde nur die Verbreitung, *nicht*, ob es hilft.
- **Helfen sie?** Gloaguen et al. testen es auf SWE-bench Lite und ihrem eigenen Benchmark CTXBench:
  - Im Vergleich zu *keiner* Kontextdatei: kein signifikanter Gesamteffekt auf die Erfolgsrate.
  - Auf CTXBench: von Entwicklern geschriebene Dateien sind signifikant besser als von einer KI generierte, aber nicht signifikant besser als gar keine Datei.
  - KI-generierte Dateien kosten mehr: **+20 %** (Lite) bzw. **+23 %** (CTXBench) Rechenkosten und **+2,45** bzw. **+3,92** Schritte im Mittel.

Anschaulich erklärt

Eine KI-generierte Kontextdatei ist wie eine lange Einarbeitungsmappe, die der Praktikant pflichtbewusst komplett liest. Er braucht dafür länger, löst die Aufgabe aber nicht häufiger. Hilfreich ist nur, was er sonst nicht herausfinden kann, z. B. „Tests startest du mit `make test-integration`“.

Merke

Erfolg, Schritte und Kosten sind **getrennte Endpunkte**. Die Empfehlung der Autoren: nur konkrete, sonst nicht verfügbare Anforderungen ergänzen. Ein optimaler Umfang von Kontext ist damit *nicht* bestimmt.

3.5 Befunde C: Verifikation, Sicherheit und Integration

Kernaussage: **Bestandene Tests belegen keine korrekte Lösung**, und auch angenommene PRs brauchen Nacharbeit.

Tests sind nur ein Näherungsmaß.

- **AutoCodeRover** (Zhang et al.): Von 78 Patches, die die Tests bestanden, waren bei manueller Prüfung nur **51 korrekt (65,4 %)**. Diese Zahl stammt aus dem *@3-Versuch* und darf nicht mit der Erfolgsrate eines einzelnen Laufs (@1) vermischt werden.
- **Aleithan et al.** (SWE-Bench+): In manchen Aufgaben steht die Lösung schon im Issue-Text oder in Kommentaren, und manche Tests sind zu schwach.
- **Wang et al.**: 877 testbestehende Patches auf SWE-bench Verified untersucht.
  - **7,8 %** scheitern, wenn man die *vollständige* Testsuite laufen lässt (SWE-bench führt nur einen Teil aus).
  - Bei **260 Patches (29,6 %)** zeigen differenzielle Tests ein anderes Verhalten als die Referenzlösung.
  - Aber: Anders ist nicht automatisch falsch. Von 77 manuell geprüften verdächtigen Patches waren **22 falsch, 4 korrekt, 51 unklar**.

Anschaulich erklärt

Tests sind wie eine Führerscheinprüfung auf einer festen Strecke. Wer sie besteht, kann auf dieser Strecke fahren, aber das heißt nicht, dass er überall sicher fährt. Eine schwache Prüfung (wenige, ungenaue Tests) lässt auch schlechte Fahrer durch.

Was nachweislich hilft (eng begrenzt).

- **Linter in SWE-agent** (Yang et al.): Mit Linter, der fehlerhafte Änderungen sofort ablehnt, werden **18 statt 15 %** der Aufgaben auf SWE-bench Lite gelöst. Das ist eine *Ablation*, also ein sauberer Einzelnachweis.
- **Cemri et al.**: Eine veränderte ChatDev-Struktur steigert den Erfolg auf 32 Aufgaben von **25,0 auf 40,6 %** (+15,6 Prozentpunkte). Aber: Es wurden *mehrere Dinge gleichzeitig* geändert, also kein isolierter Beweis, dass eine abschließende Zielprüfung wirkt.

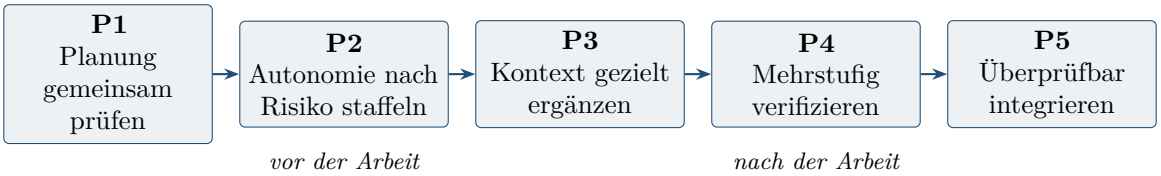
Sicherheit ist ein eigener Endpunkt. Sajadi et al. finden Sicherheitslücken in Patches mehrerer Agenten. Weil sich Modelle und Bearbeitungsmechanismen unterscheiden, kann man das **nicht ursächlich dem Autonomiegrad zuschreiben**. Bei Llama-Patches hängen mehr geänderte Dateien mit mehr Sicherheitsproblemen zusammen, dass kleinere Patches schützen, wurde aber nicht gemessen.

Integration in echte Projekte.

- **Watanabe et al.**: 475 von 567 Claude-Code-PRs wurden gemergt, davon **214 (45,1 %)** vorher überarbeitet. Häufigster Grund: Bugfixes (47,7 %). Der Umfang der Überarbeitung unterscheidet sich nicht signifikant von menschlichen PRs, aber das beweist nicht, dass der Aufwand gleich ist.
- **Li et al.**: Angenommene Codex-PRs werden schneller geschlossen. Das misst aber weder Arbeitszeit noch Durchsatz. Bei 58,2 % der Agenten-PRs (und 75,3 % der menschlichen) gibt es keine sichtbare Review-Aktion auf GitHub, wie gründlich geprüft wurde, ist also unbekannt.

3.6 Die fünf Gestaltungsprinzipien

Wichtig vorab: Die Prinzipien verbinden die Befunde zu einem *vorgeschlagenen* Workflow. Jede „Vorschlag“-Regel ist eine **eigene Empfehlung, keine belegte Wirkung**.



Prinzip	Handlungsregel (Vorschlag)	Woher kommt die Idee?	Was ist *nicht* belegt?
P1 Planung gemeinsam prüfen	Bei unklaren Anforderungen Ziel und Akzeptanzkriterien vor der Umsetzung abstimmen	HULA und Interviewstudie zeigen: so eine Beteiligung ist praktisch machbar	Zeit- oder Qualitätseffekt; ein Plan ersetzt kein Code-Review
P2 Autonomie nach Risiko und Prüfbarkeit staffeln	Sicherheitskritische oder schwer rückgängig zu machende Änderungen menschlich freigeben	Sicherheits- und Aufsichtsbefunde; OpenHands-Sandbox	Kausaler Effekt; Sandbox macht den Code nicht sicher
P3 Kontext gezielt ergänzen	Fehlende Build-, Test- und Projektanforderungen gezielt ergänzen	Bestandsanalysen und Kontextversuche	Dass kürzere Dateien die Erfolgsrate allgemein steigern
P4 Mehrstufig und am Auftrag verifizieren	Bestandene Tests durch zusätzliche, anforderungsbezogene Prüfungen ergänzen	Linter-Ablation, Wang et al., Cemri et al.	Dass die Kombination menschliche Nacharbeit senkt
P5 Überprüfbar integrieren	Änderungen sachlich abgrenzen und vor dem Merge nachvollziehbar prüfen	Revisions- und Sicherheitsbefunde	Optimale PR-Größe; Nettozeitgewinn

P4 genauer: statische Prüfungen (z. B. Linter) + aufgabenbezogene Tests + vor der Integration die *vollständige* Testsuite. Zusätzliche Tests müssen die Anforderung über die schon bestandenen Fälle hinaus prüfen, und ob der Gesamtauftrag erfüllt ist, wird gesondert beurteilt.

P5 genauer: kleine, sachlich abgegrenzte PRs, Beteiligung des Agenten dokumentiert, menschliches Review und Sicherheitsprüfung je nach Risiko.

3.7 Diskussion: Was trägt die Evidenz?

- Die **stärksten Belege** gibt es für eng begrenzte Maßnahmen (z. B. Linter).
- **Nutzerexperimente** zeigen: Weniger aktive Arbeit garantiert keine kürzere Gesamtdauer.
- **Repository-Analysen** sind praxisnah, zeigen aber keine Ursachen, weil Aufgaben und Auswahl sich unterscheiden.
- Akzeptanz, Testbestehen und Änderungsumfang **ersetzen nicht** die Messung von Wartbarkeit, Sicherheit oder Zeit.

Grenzen der eigenen Arbeit. Narrative Auswahl (nicht vollständig, nicht voll reproduzierbar); wichtige Befunde stammen aus Preprints; Modelle, Agentenversionen und Budgets unterscheiden sich; SWE-bench enthält nur Python-Projekte; 17 Interviews sind keine repräsentative Häufigkeitsschätzung.

Der zentrale Zielkonflikt. Mehr Prüfen kostet Zeit und Geld. Es lohnt sich nur, wenn die zusätzlichen Prüfschritte **genug Fehler finden**, um den Aufwand zu rechtfertigen.

Vorgeschlagener nächster Forschungsschritt. Ein kontrollierter Vergleich **mit und ohne menschliche Planfreigabe**, bei gleichem Agenten, gleichem Modell und gleichem Budget. Getrennt gemessen: aktive menschliche Zeit, Gesamtdauer, Inferenzkosten, Review-Aufwand, funktionale Fehler und Sicherheitsbefunde; später ergänzt um Wartungsaufgaben.

3.8 Fazit

Die Antwort auf die Forschungsfrage

Teil 1: Aus der Literatur lassen sich fünf begründete Prinzipien ableiten.
Teil 2: Für einzelne, eng begrenzte Maßnahmen gibt es kontrollierte Belege. Die Wirkung der fünf Prinzipien als Gesamtworkflow und ihre Übertragbarkeit sind nicht nachgewiesen.

Also: **Konkrete Empfehlungen sind möglich, ein allgemeiner Gewinn an Codequalität oder Produktivität folgt daraus nicht.** Dafür braucht es vergleichbare Endpunkte sowie kontrollierte und langfristige Studien.

4 Die wichtigsten Studien verständlich erklärt

Hier stehen die Studien, die auf den Folien vorkommen, ausführlich. Die übrigen folgen danach kurz.

4.1 Motivation und Hintergrund

Robbes et al. (2026): „Agentic much?“ ACM TOSEM

Was wurde gemacht? 128.018 GitHub-Projekte nach Spuren von Agenten durchsucht, etwa Konfigurationsdateien oder Co-Author-Einträge in Commits.
Was kam heraus? 22,20 % der Projekte nutzen Agenten (vorsichtig), bis zu 28,66 % (hochgerechnet).
Was heißt das für unser Paper? Belegt, dass das Thema praktisch relevant ist (Einleitung).
Grenzen: Agenten ohne Spuren werden übersehen; eine Konfigurationsdatei heißt noch keine intensive Nutzung; nicht repräsentativ für die gesamte Softwareentwicklung.

Takerngsaksiri et al. (2025): HULA ICSE-SEIP, Atlassian

Was wurde gemacht? Agent in Atlassians JIRA eingebaut: Ein Planungs-Agent macht einen Plan, der Mensch prüft ihn, dann schreibt ein Coding-Agent Code, den der Mensch wieder prüft.
Was kam heraus? Im Einsatz wurden 82 % der Pläne freigegeben; von den daraus entstandenen PRs wurden 59 % gemergt. Detaillierte Aufgabenbeschreibungen sind wichtig.
Was heißt das für unser Paper? Zeigt, dass gemeinsame Planung praktisch funktioniert (Grundlage für P1).
Grenzen: Ein Unternehmen, Zeitersparnis nur per Umfrage, Autoren aus der Firma.

Dhanorkar et al. (2026): Aufsicht in der Praxis — arXiv-Preprint, Microsoft

Was wurde gemacht? 17 erfahrene Entwickler interviewt, wie sie Agenten beaufsichtigen.
Was kam heraus? Vier Formen der Aufsicht: Vorgaben vorab, gemeinsame Planung, laufende Überwachung, nachträgliches Review. Beim Prüfen nutzen Entwickler Abkürzungen (Heuristiken), etwa: dem Plan vertrauen, grüne Tests als Beweis nehmen, nur überfliegen.
Was heißt das für unser Paper? Liefert das Vokabular für Aufsicht und stützt P1 und P2. Zeigt: Ein Plan ist keine Garantie.
Grenzen: Qualitativ, klein, keine Messung von Fehlerraten oder Wirkungen.

4.2 Produktivität

Chen et al. (2026): „Code with me or for me?“ — CHI '26

Was wurde gemacht? 20 Studierende lösen realistische Programmieraufgaben einmal mit Copilot und einmal mit dem Agenten OpenHands.
Was kam heraus? Mit OpenHands 60 % statt 25 % korrekt gelöst. Aktive Zeit bei erfolgreichen Aufgaben 12,5 statt 25,1 min, mit Laufzeit des Agenten aber 27,9 min.
Was heißt das für unser Paper? Kernbeleg für „weniger aktive Arbeit ist nicht kürzere Gesamtdauer“.
Grenzen: Kleine Gruppe von Studierenden; Erfolgsquoten unterscheiden sich, daher Zeitvergleich schwierig; Autoren aus dem OpenHands-Umfeld.

Becker et al. (2025): METR-Studie — arXiv-Preprint

Was wurde gemacht? 16 erfahrene Open-Source-Entwickler bearbeiten 246 echte Aufgaben in ihren eigenen Projekten. Per Zufall wird entschieden, ob KI erlaubt ist (meist Cursor).
Was kam heraus? Mit KI dauerten Aufgaben 19 % länger. Die Entwickler glaubten danach aber, 20 % Zeit gespart zu haben.
Was heißt das für unser Paper? Starker Gegenbefund: Gefühlte Produktivität ist nicht gemessene Produktivität.
Grenzen: Sehr spezielle Gruppe (Experten in vertrauten Projekten), kleine Stichprobe, Werkzeuge von Anfang 2025.

Cui et al. (2026): Drei Feldexperimente — Management Science

Was wurde gemacht? In drei Unternehmen bekamen 4.867 Entwickler per Zufall Zugang zu Copilot-Codevervollständigung.
Was kam heraus? 26,08 % mehr erledigte Aufgaben (gemessen als PRs pro Woche), Standardfehler 10,3 %.
Was heißt das für unser Paper? Zeigt, dass ein anderes Werkzeug und ein anderes Maß zu ganz anderen Zahlen führen.
Grenzen: Kein Agent! Nur als angrenzender Vergleich im Paper.

4.3 Kontext

Gloaguen et al. (2026): „Evaluating AGENTS.md“ — ICLR-2026-Workshop

Was wurde gemacht? Agenten lösen Aufgaben in drei Varianten: ohne Kontextdatei, mit KI-generierter Datei, mit von Entwicklern geschriebener Datei. Getestet auf SWE-bench Lite und dem eigenen Benchmark CTXBench.
Was kam heraus? Kein signifikanter Gesamteffekt auf die Erfolgsrate. KI-generierte Dateien erhöhen Kosten (+20/+23 %) und Schritte (+2,45/+3,92).
Was heißt das für unser Paper? Grundlage für P3: nur gezielt ergänzen.
Grenzen: Misst nur die Erfolgsrate, nicht, ob Konventionen eingehalten werden oder der Code wartbarer wird.

4.4 Verifikation und Integration

Jimenez et al. (2024): SWE-bench — ICLR

Was wurde gemacht? Benchmark aus echten GitHub-Issues in 12 Python-Projekten; Prüfung über die Tests des Original-Fixes.
Was kam heraus? Wurde zum Standardtest für Coding-Agenten.
Was heißt das für unser Paper? Fast alle technischen Befunde im Paper beruhen darauf.
Grenzen: Nur Python, nur populäre Projekte; Qualität hängt an den Tests.

Zhang et al. (2024): AutoCodeRover — ISSTA

Was wurde gemacht? Agent, der Code als Programmstruktur durchsucht und Issues repariert. Testbestehende Patches wurden zusätzlich von Hand geprüft.
Was kam heraus? Nur 51 von 78 testbestehenden Patches waren wirklich korrekt (65,4 %, im @3-Versuch).
Was heißt das für unser Paper? Zeigt: Testbestehen $\neq$ Korrektheit.
Grenzen: @3-Wert nicht mit @1-Erfolgsrate verwechseln; nur Benchmark.

Wang, Pradel, Liu (2026): „Solved issues really solved?“ — ICSE

Was wurde gemacht? 877 testbestehende Patches von Agenten auf SWE-bench Verified nachgeprüft: mit voller Testsuite und mit differenziellen Tests gegen die Referenz.
Was kam heraus? 7,8 % scheitern an der vollen Testsuite; 29,6 % verhalten sich anders als die Referenz; von 77 manuell geprüften: 22 falsch, 4 korrekt, 51 unklar.
Was heißt das für unser Paper? Grundlage für P4: volle Testsuite und zusätzliche Prüfungen gegen die Anforderung.
Grenzen: Abweichung ist kein Beweis für einen Fehler; nur Python.

Yang et al. (2024): SWE-agent — NeurIPS

Was wurde gemacht? Agent mit einer speziell für Sprachmodelle gestalteten Werkzeugschnittstelle. In einer Ablation wurde der Linter weggelassen.
Was kam heraus? Mit Linter 18 %, ohne 15 % gelöste Aufgaben auf SWE-bench Lite.
Was heißt das für unser Paper? Stärkster sauberer Einzelbeleg im Paper (P4).
Grenzen: Nur Benchmark, kleiner Effekt, ältere Modelle.

Cemri et al. (2025): „Why do multi-agent LLM systems fail?“ NeurIPS, Datasets & Benchmarks

Was wurde gemacht? Fehler in Systemen mit mehreren Agenten analysiert und eine Fehlertaxonomie (MAST) erstellt; außerdem Eingriffe getestet.
Was kam heraus? Veränderte ChatDev-Struktur: 25,0 → 40,6 % auf 32 Aufgaben.
Was heißt das für unser Paper? Stützt P4 (Erfüllung des Gesamtauftrags gesondert prüfen).
Grenzen: Mehrere Änderungen gleichzeitig, kein isolierter Effekt; kleine Programme.

Watanabe et al. (2026): Claude-Code-PRs auf GitHub ACM TOSEM

Was wurde gemacht? 567 mit Claude Code erstellte PRs aus Open-Source-Projekten ausgewertet.
Was kam heraus? 475 gemergt, davon 214 (45,1 %) vorher überarbeitet, am häufigsten wegen Bugfixes.
Was heißt das für unser Paper? Grundlage für P5: Review und Nacharbeit bleiben nötig.
Grenzen: Nur ein Werkzeug, nur öffentlich erkennbare PRs.

4.5 Weitere Arbeiten im Korpus (kurz)

Arbeit	Rolle im Paper
Liu et al. (2024), Survey	Definition des Coding-Agenten (Planung, Gedächtnis, Wahrnehmung, Aktion).
Xia et al. (2025), Agentless	Beispiel für geringe Autonomie: fester Ablauf aus Lokalisieren, Reparieren, Validieren.
Wang et al. (2025), OpenHands	Plattform mit Docker-Sandbox; Agent in mehreren anderen Studien.
Chatlatanagulchai et al. (2025)	253 `CLAUDE.md`-Dateien: was typischerweise drinsteht.
Mohsenimofidi et al. (2026)	466 von 10.000 Projekten mit Kontextdatei; nur Verbreitung.
Aleithan et al. (2024), SWE-Bench+	Lösungen im Issue-Text und schwache Tests als Benchmark-Risiken.
Sajadi et al. (2025)	Sicherheitslücken in Agenten-Patches; keine kausale Aussage zum Autonomiegrad.
Li et al. (2025), AIDev	Schnelle Schließzeiten, häufig keine sichtbare Review-Aktion.

5 Wie hängen Befunde und Prinzipien zusammen?

Befund		führt zu
Aufsicht hat mehrere Formen; Plan als Prüfhilfe ist riskant (Dhanorkar); Planfreigabe ist machbar (HULA)	→	**P1**: Ziel und Akzeptanzkriterien vorab klären, Review trotzdem nötig
Sicherheitslücken in Agenten-Patches (Sajadi); Sandbox schützt nur das System (OpenHands)	→	**P2**: Kritisches und Unumkehrbares menschlich freigeben
Kontextdateien: keine messbare Erfolgssteigerung, aber mehr Kosten (Gloaguen)	→	**P3**: nur Fehlendes gezielt ergänzen
Tests übersehen Fehler (AutoCodeRover, Wang); Linter hilft (SWE-agent)	→	**P4**: mehrstufig prüfen, volle Testsuite, Zusatztests
45 % der gemergten Agenten-PRs überarbeitet (Watanabe); Sicherheitsrisiken	→	**P5**: kleine PRs, nachvollziehbares Review

6 Typische Denkfehler und wie man es richtig sagt

Falsch	Richtig
„Agenten machen 35 % produktiver.“	Erfolgsquote +35 Prozentpunkte (25 → 60 %), bei 20 Studierenden im Labor.
„Agenten halbieren die Arbeitszeit.“	Die *aktive* Zeit halbiert sich; mit Agentenlaufzeit 27,9 min.
„Kontextdateien bringen nichts.“	Kein *signifikanter* Gesamteffekt auf die Erfolgsrate; das ist kein Beweis gleicher Leistung.
„29,6 % der Patches sind falsch.“	29,6 % verhalten sich anders; von 77 geprüften waren 22 falsch, 51 unklar.
„Mehr Autonomie erzeugt mehr Sicherheitslücken.“	Lücken bei mehreren Agenten, aber keine kausale Zuschreibung zum Autonomiegrad möglich.
„Eine Zielprüfung bringt +15,6 %.“	+15,6 Prozentpunkte durch mehrere Änderungen gleichzeitig.
„Eine Sandbox macht den Agenten sicher.“	Sie schützt das System, nicht die Qualität des erzeugten Codes.
„Unsere Prinzipien verbessern die Qualität.“	Begründete Empfehlungen; Wirkung als Gesamtworkflow nicht nachgewiesen.
„Wir haben eine systematische Review gemacht.“	Narrative Analyse; Trefferzahlen nicht berichtet.

7 Prüfungsfragen mit Antworten

Zur Methodik

Warum keine systematische Literaturanalyse? Für ein Short Paper wurde gezielt narrativ ausgewählt. Suche, Begriffe und Kriterien sind dokumentiert, die Trefferzahlen pro Schritt aber nicht, deshalb beanspruchen wir keine reproduzierbare Vollerhebung. Das ist im Paper offen als Grenze benannt.

Warum keine gemeinsame Effektgröße über alle Studien? Die Studien messen Verschiedenes (Erfolgsrate, aktive Zeit, Gesamtdauer, PRs pro Woche, Kosten), mit unterschiedlichen Modellen, Budgets und Personen. Eine Zahl daraus wäre irreführend. Deshalb sortieren wir nach Evidenzart.

Viele Quellen sind Preprints. Ist das ein Problem? Ja, das ist eine Einschränkung. Das Feld ist sehr schnell, wichtige Nutzer- und Sicherheitsbefunde (Becker, Dhanorkar, Sajadi) liegen nur als Preprint vor. Wo es eine begutachtete Fassung gibt, zitieren wir diese, und der Status ist bei der Gewichtung zu beachten.

Warum ist Cui drin, obwohl es um Codevervollständigung geht? Bewusst nur als angrenzender Vergleich. Die Studie zeigt, dass Werkzeugtyp und Messgröße das Ergebnis stark prägen, und das steht so auch in der Tabelle.

Gibt es Interessenkonflikte? Ja: Chen et al. stammen aus dem OpenHands-Umfeld, HULA von Atlassian, Dhanorkar et al. von Microsoft. Das spricht für vorsichtige Einordnung. (Im Paper nicht eigens genannt, kann man ehrlich ergänzen.)

Sind die Ergebnisse bei den schnellen Modellfortschritten noch aktuell? Absolute Zahlen veralten schnell, weil viele Studien Modelle von 2024 oder Anfang 2025 nutzen. Die methodischen Lehren bleiben: Testbestehen ist nicht Korrektheit, aktive Zeit ist nicht Gesamtdauer.

Zum Inhalt

Widersprechen sich Cui (+26 %) und Becker (19 % langsamer)? Nein, nicht direkt. Anderes Werkzeug (Vervollständigung vs. Chat/Agent), andere Personen (breite Belegschaft vs. Experten im eigenen Projekt), anderes Maß (PRs pro Woche vs. Bearbeitungszeit).

Spart der Agent bei Chen Zeit? Die aktive Zeit halbiert sich ungefähr, die Gesamtdauer nicht (27,9 vs. 25,1 min). Außerdem gelten die Zeiten nur für erfolgreiche Aufgaben, und die Erfolgsquoten unterscheiden sich.

Warum glaubten die Entwickler bei Becker, schneller zu sein? Selbsteinschätzungen sind unzuverlässig. Die Autoren nennen u. a. Optimismus, die hohe Vertrautheit mit dem eigenen Projekt, große Codebasen, wenig zuverlässige KI-Ausgaben und fehlendes implizites Projektwissen. Folge: Messen statt fragen.

Soll man keine `CLAUDE.md` schreiben? Nicht pauschal. Es gibt keinen signifikanten Gesamteffekt auf die Erfolgsrate, aber höhere Kosten bei KI-generierten Dateien. Deshalb P3: nur ergänzen, was der Agent sonst nicht findet. Ob Konventionen besser eingehalten werden, wurde nicht gemessen.

Warum reichen grüne Tests nicht? Tests prüfen nur, was sie abdecken. AutoCodeRover: 51 von 78 korrekt. Wang: 7,8 % scheitern an der vollen Suite, 29,6 % verhalten sich anders. Darum P4.

Ist weniger Autonomie also besser (Agentless)? Agentless zeigt, dass ein fester Ablauf konkurrenzfähig sein kann. Ein sauberer Autonomieeffekt folgt nicht, weil sich auch Modell und Budget unterscheiden. Deshalb empfehlen wir *risikogerechte*, nicht minimale Autonomie.

Warum ist der Linter-Befund stärker als der Cemri-Befund? Beim Linter wurde nur ein Teil verändert (Ablation), also ist der Effekt dem Linter zuzuschreiben. Bei Cemri wurden mehrere Dinge zugleich geändert.

Schnell geschlossene Agenten-PRs (Li): ist das Produktivität? Nein. Die Schließzeit misst weder Arbeitszeit noch Durchsatz, und oft fehlt eine sichtbare Review-Aktion. Über die Prüftiefe sagt das nichts.

Warum kleine PRs, wenn Agenten-Commits in der Praxis eher groß sind? Gerade deshalb: Kleine, abgegrenzte Änderungen lassen sich nachvollziehbar prüfen. Die optimale Größe hat aber niemand bestimmt.

Transfer

Was würdet ihr einem Unternehmen raten? Die fünf Handlungsregeln anwenden und vor allem den Nutzen im eigenen Team messen: aktive Zeit, Gesamtdauer, Review-Aufwand und Fehler getrennt, statt sich auf Gefühl oder Benchmarks zu verlassen.

Wie würdet ihr den vorgeschlagenen Versuch aufbauen? Gleiche Aufgaben, gleicher Agent, gleiches Modell und Budget, zufällige Zuteilung zu „mit“ oder „ohne“ Planfreigabe. Getrennt erfassen: aktive Zeit, Gesamtdauer, Kosten, Review-Aufwand, funktionale Fehler, Sicherheitsbefunde; später Wartungsaufgaben am selben Code.

Ist „Evidenzgrenzen“ nicht ein schwaches Ergebnis? Nein. Gerade bei einem Hype-Thema ist es ein Beitrag zu zeigen, welche Aussagen die Literatur trägt und welche Messgrößen künftige Studien trennen müssen.

Und wenn man eine Frage nicht beantworten kann? Ehrlich sagen, dass die Studie das nicht gemessen hat oder dass es außerhalb des Korpus liegt, und auf eine Kernaussage zurückführen. Das passt zur Grundhaltung des Papers.

8 Alle wichtigen Zahlen auf einen Blick

Zahl	Bedeutung	Quelle
21	ausgewertete Arbeiten (2024–2026)	Paper
128.018 • 22,20 % • 28,66 %	Projekte • Adoption vorsichtig • obere Schätzung	Robbes
25 → 60 %	Erfolg Copilot → OpenHands	Chen
25,1 → 12,5 min • 27,9 min	aktive Zeit • inklusive Agentenlaufzeit	Chen
+26,08 % (SE 10,3 %)	erledigte Aufgaben, 4.867 Entwickler, Vervollständigung	Cui
+19 % • 20 % gefühlt	längere Bearbeitungszeit • geschätzte Ersparnis; 16 Personen, 246 Aufgaben	Becker
17	Interviews	Dhanorkar
253	`CLAUDE.md`-Dateien	Chatlatanagulchai
466 von 10.000	Projekte mit Kontextdatei	Mohsenimofidi
+20 / +23 % • +2,45 / +3,92	Kosten • Schritte (Lite / CTXBench)	Gloaguen
51 von 78 = 65,4 %	korrekte testbestehende Patches (@3)	AutoCodeRover
877 • 7,8 % • 29,6 %	Patches • scheitern an voller Suite • Verhaltensabweichung	Wang 2026
77 → 22 / 4 / 51	geprüft → falsch / korrekt / unklar	Wang 2026
18 vs. 15 %	mit / ohne Linter	SWE-agent
25,0 → 40,6 %	ChatDev-Umbau, 32 Aufgaben	Cemri
475 / 567 • 214 = 45,1 %	gemergt • davon überarbeitet	Watanabe
58,2 % / 75,3 %	ohne sichtbare Review-Aktion: Agent / Mensch	Li

Merke

Wenn du nur drei Dinge behältst:

1. Weniger aktive Arbeit ist keine kürzere Gesamtdauer.
2. Bestandene Tests sind keine korrekte Lösung.
3. Die Prinzipien sind begründete Empfehlungen, keine bewiesenen Wirkungen.